Sensors & Systems — Exercise Notes

Lecture 5 Exercise 1: 1D Kalman Filter for IMU

Jan Bendtsen, Torben Knudsen | Electronic Systems, 8th Semester

Exercise Statement

Implement the Kalman filter for the 1D IMU (perfect model/estimator match) shown in the slides with:

$$\phi = 0.9, \quad \sigma_{w_a} = \sigma_{w_b} = \sigma_\nu = 0.1, \quad b = -0.2, \quad T_s = 0.001$$

Replicate the figures on Slide 25 (see Figure 1).

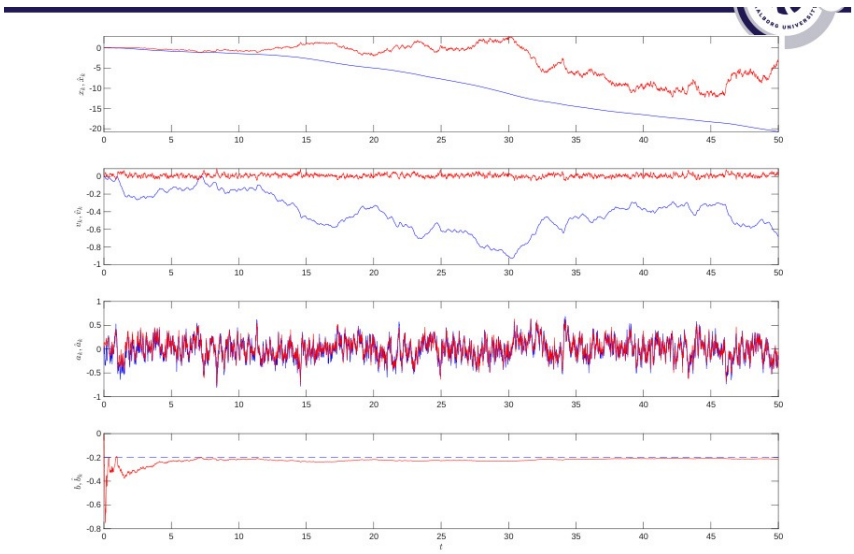

Figure 1: Slide 25 reference result. **Blue** = true signal; **Red** = KF estimate. From top to bottom: position x_k, velocity v_k, acceleration a_k, bias b_k. Time axis $t = 0 \ldots 50\,\mathrm{s}$.

A full IMU has three sensor types - this exercise uses only one.
The gyroscope and magnetometer are essential for 3-D attitude (roll, pitch, yaw) estimation. That problem is nonlinear (trigonometric functions of the current orientation appear in the equations), which forces us to use the Extended or Unscented KF. In this 1-D exercise the system is linear throughout — the standard linear KF applies directly.

Stochastic errors — tracked by the Kalman filter

White noise $\boldsymbol{\nu}$
Origin: thermo-mechanical fluctuations inside the MEMS structure (Brownian motion of the proof mass).
Model: $\nu_k \sim \mathcal{N}(0,\, \sigma_\nu^2)$, independent sample to sample.

Key property: each sample is equally likely to be above or below the true value. The mean is zero *and* stays zero at every instant — because consecutive samples are independent, there is no accumulation.

Bias random walk $\boldsymbol{\epsilon}$
Origin: **flicker noise** $(1/f)$ in the analogue readout — slow fluctuations that drift the DC operating point. Model: $b_{k+1} = b_k + \sigma_{w_b}\, \bar{w}_{b,k}, \quad \bar{w}_{b,k} \sim \mathcal{N}(0, 1)$.

Key property: each increment is zero-mean, but the bias accumulates all past increments, so its variance grows as $\sigma_{w_b}^2 k$ — it wanders without bound and cannot be calibrated away.

In the exercise: $\sigma_{w_b} = 0.1$.

Deterministic errors — remove by calibration

Accelerometer:

$$K_a \tilde{\mathbf{a}} = (I + S_a + M_a)\, \mathbf{a} + \mathbf{b}_a + T_a \Delta T + \boldsymbol{\nu}_a + \boldsymbol{\epsilon}_a$$

Gyroscope:

$$K_g \tilde{\boldsymbol{\omega}} = (I + S_g + M_g)\, \boldsymbol{\omega} + \mathbf{b}_g + G_g\, \mathbf{a} + T_g \Delta T + \boldsymbol{\nu}_g + \boldsymbol{\epsilon}_g$$

Deterministic errors — remove by calibration

Symbol	Name	What it does
K_a, K_g	Sensitivity	Overall gain error (volts per $\mathrm{m/s^2}$); single scalar per axis.
S_a, S_g	Scale factor distortion	Each axis has its own gain error: x reads $1.02\,g$, y reads $0.97\,g$. Diagonal matrix $\mathrm{diag}(S_x, S_y, S_z)$.
M_a, M_g	Misalignment	Axes not exactly 90 apart, could be y and z (**cross-axis coupling**). Removed by the **tumble test** on a precision turntable.
$G_g\mathbf{a}$	g-sensitivity (gyro only)	Linear acceleration deflects the gyro proof mass, faking a rotation signal. Significant only in high-g manoeuvres.
$T_a\Delta T$, $T_g\Delta T$	Temperature drift	MEMS spring stiffness shifts with temperature, moving the bias. Corrected at runtime via an on-chip thermometer and lookup table.

The 4-State Model - Exercise 1

State vector

$$\boldsymbol{\chi}_k = \begin{bmatrix} x_k \\ v_k \\ a_k \\ b_k \end{bmatrix} \qquad \begin{aligned} &x_k : \text{position [m]} \\ &v_k : \text{velocity [m/s]} \\ &a_k : \text{true acceleration [m/s}^2\text{]} \\ &b_k : \text{accelerometer bias [m/s}^2\text{]} \end{aligned}$$

where the bias evolves as a random walk,

$$b_{k+1} = b_k + \sigma_{w_b}\,\bar{w}_{b,k}, \qquad \bar{w}_{b,k} \sim \mathcal{N}(0,1),$$

i.e. each step the bias shifts by a zero-mean Gaussian increment with standard deviation σ_{w_b}.

System matrices

State equation: $\boldsymbol{\chi}_{k+1} = \boldsymbol{\Phi}\,\boldsymbol{\chi}_k + Q^{1/2}\bar{\mathbf{w}}_k$ Each row of $\boldsymbol{\Phi}$ encodes one update rule:

$$\underbrace{\begin{bmatrix} x_{k+1} \\ v_{k+1} \\ a_{k+1} \\ b_{k+1} \end{bmatrix}}_{\boldsymbol{\chi}_{k+1}} = \underbrace{\begin{bmatrix} 1 & T_s & 0 & 0 \\ 0 & 1 & T_s & 0 \\ 0 & 0 & \phi & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}}_{\boldsymbol{\Phi},\ \text{state transition}} \underbrace{\begin{bmatrix} x_k \\ v_k \\ a_k \\ b_k \end{bmatrix}}_{\boldsymbol{\chi}_k} + \underbrace{\begin{bmatrix} 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & \sigma_{w_a}^2 & 0 \\ 0 & 0 & 0 & \sigma_{w_b}^2 \end{bmatrix}^{1/2}}_{Q,\ \text{process noise}} \underbrace{\begin{bmatrix} 0 \\ 0 \\ \bar{w}_{a,k} \\ \bar{w}_{b,k} \end{bmatrix}}_{\bar{\mathbf{w}}_k}$$

$$\begin{aligned} &\leftarrow x_{k+1} = x_k + T_s v_k \quad \text{(Euler position)} \\ &\leftarrow v_{k+1} = v_k + T_s a_k \quad \text{(Euler velocity)} \\ &\leftarrow a_{k+1} = \phi\, a_k + \sigma_{w_a}\bar{w}_{a,k} \quad \text{(AR(1))} \\ &\leftarrow b_{k+1} = b_k + \sigma_{w_b}\bar{w}_{b,k} \quad \text{(random walk)} \end{aligned}$$

- $Q_{11} = Q_{22} = 0$ (no direct noise becuase not measured, driven only through $\boldsymbol{\Phi}$)

Measurement equation: $y_k = H\,\boldsymbol{\chi}_k + R^{1/2}\bar{\nu}_k$

$$\underbrace{\begin{bmatrix} y_k \end{bmatrix}}_{y_k} = \underbrace{\begin{bmatrix} 0 & 0 & 1 & 1 \end{bmatrix}}_{H,\ \text{measurement matrix}} \underbrace{\begin{bmatrix} x_k \\ v_k \\ a_k \\ b_k \end{bmatrix}}_{\boldsymbol{\chi}_k} + \underbrace{\begin{bmatrix} \sigma_\nu^2 \end{bmatrix}^{1/2}}_{R,\ \text{measurement noise}} \underbrace{\begin{bmatrix} \bar{\nu}_k \end{bmatrix}}_{\bar{\nu}_k} \qquad \leftarrow y_k = x_k + \sigma_\nu \bar{\nu}_k \quad \text{(position measurement)}$$

The zeros in positions 1 and 2 of H say: **the accelerometer does not measure position or velocity** — not directly, not indirectly. This is why x and v are unobservable and their estimates drift.

> The $Q^{1/2}$ and $R^{1/2}$ notation keeps the random variables $\bar{w}_k \sim \mathcal{N}(0, I)$ and $\bar{\nu}_k \sim \mathcal{N}(0,1)$ as standard normals (variance 1, unit-free).
>
> The scaling factor carries all the units: $\mathrm{Var}(R^{1/2}\bar{\nu}_k) = R = \sigma_\nu^2$ and $\mathrm{Var}(Q^{1/2}\bar{w}_k) = Q$.
>
> In the Kalman filter equations only Q and R appear — never the square roots.

Exercise 1 Results

The given parameters:

$$\phi = 0.9, \quad \sigma_{w_a} = \sigma_{w_b} = \sigma_\nu = 0.1, \quad b = -0.2, \quad T_s = 0.001$$

caused difficulty replicating slide 25, due to two consequences of $\sigma_{w_b} = 0.1$.

1. Bias never converges. Each prediction step re-inflates the bias uncertainty:

$$P^{[b,b]}_{k+1|k} = P^{[b,b]}_{k|k} + \sigma^2_{w_b}$$

which competes with the measurement update that reduces $P^{[b,b]}$. At steady state the two balance, leaving $P^{[b,b]}_\infty > 0$, so $K^{[b]}$ never reaches zero and the bias estimate keeps oscillating rather than locking in. (because $\sigma_{w_b} > 0$)

2. Acceleration also degrades. With $\sigma_{w_b} = \sigma_{w_a}$, the bias row of the Kalman gain:

$$K^{[b]}_k = \frac{P^{[b,x]}_{k|k-1}}{S_k}, \qquad S_k = P^{[x,x]}_{k|k-1} + \sigma^2_\nu$$

is comparable in magnitude to $K^{[a]}_k$, so innovations are split equally between $\hat{a}$ and $\hat{b}$. This grows the cross-covariance $P^{[a,b]} \neq 0$, meaning the filter **cannot separate acceleration from bias** — correcting one always nudges the other. When $\sigma_{w_b} \ll \sigma_{w_a}$ the model says bias barely moves, innovations are attributed mostly to $\hat{a}$, $P^{[a,b]}$ stays small, and acceleration tracks cleanly.

Results

σ_{w_b}	$\sigma^2_{w_b}$	Behaviour
0.1	0.01 (equal to R)	$K^{[b]}$ large at steady state; bias updated aggressively every step — oscillates wildly.
0.01	10^{-4}	$K^{[b]}$ smaller but significant; bias bounces continuously, never settles.
0.001	10^{-6}	$K^{[b]}$ very small; bias hovers near -0.2 with small persistent oscillations.
0	0	$P^{[b,b]}$ only decreases; $K^{[b]} \to 0$; bias **locks in** to -0.2.

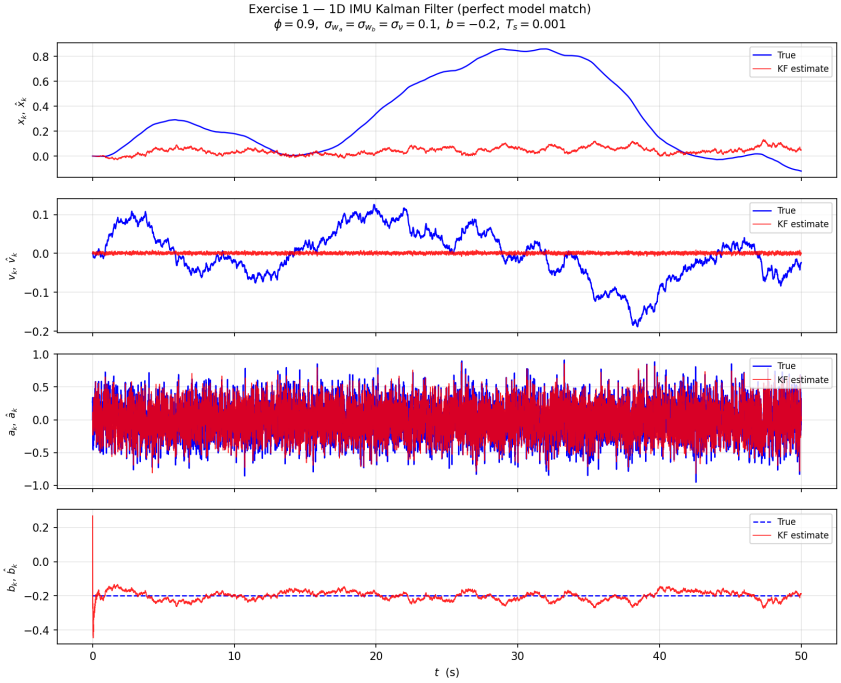

Figure 2: $\sigma_{w_b} = 0.001$: bias estimate hovers around -0.2 with small persistent oscillations. Because $Q_{33} = 10^{-6} > 0$, the covariance prediction re-inflates $P^{[b,b]}$ every step, keeping $K^{[b]}$ non-zero and preventing the estimate from locking in.

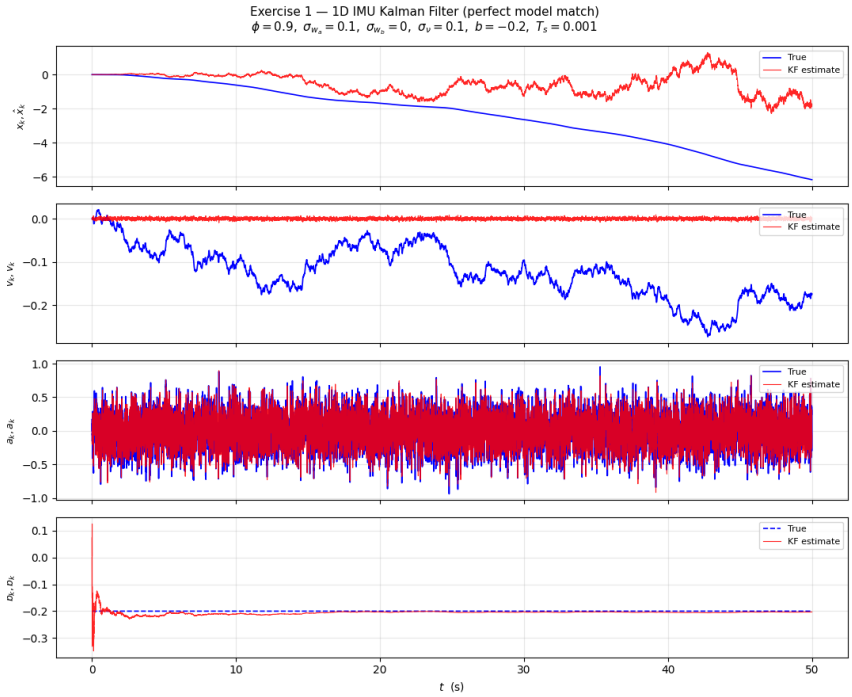

Figure 3: Python simulation output. **Blue** = true signal; **Red** = KF estimate. Parameters: $\phi = 0.9$, $\sigma_{w_a} = \sigma_\nu = 0.0$,$\sigma_{w_b} = 0.0$, $b = -0.2$, $T_s = 0.001$, $N = 50\,000$ steps (50 s).

Exercise 2 Acceleration as External Input (Slide 27)

Set acceleration as a known external input, and combine it with the Kalman filter using the same parameter values. Replicate the figures on Slide 27.

Two-model approach

This exercise uses **two different models** running in parallel — one for the true physical system and one inside the Kalman filter.

True system — 3 states $[x, v, a]$:

$$\underbrace{\begin{bmatrix} x_{k+1} \\ v_{k+1} \\ a_{k+1} \end{bmatrix}}_{\boldsymbol{\chi}_{\text{true},k+1}} = \underbrace{\begin{bmatrix} 1 & T_s & 0 \\ 0 & 1 & T_s \\ 0 & 0 & 0 \end{bmatrix}}_{\boldsymbol{\Phi}_{\text{true}},\ \text{state transition}} \underbrace{\begin{bmatrix} x_k \\ v_k \\ a_k \end{bmatrix}}_{\boldsymbol{\chi}_{\text{true},k}} + \underbrace{\begin{bmatrix} 0 \\ 0 \\ 1 \end{bmatrix}}_{\boldsymbol{\Gamma}} \underbrace{a_k^{\text{input}}}_{\text{known input}} \qquad \begin{aligned} &\leftarrow x_{k+1} = x_k + T_s v_k \\ &\leftarrow v_{k+1} = v_k + T_s a_k \\ &\leftarrow a_{k+1} = a_k^{\text{input}} \end{aligned}$$

Acceleration is a *known* deterministic input a_k^{input}, not a random state.

Kalman filter — 4 states $[x, v, a, b]$ (Slide 23/25 model):

The filter keeps 4 states with $\sigma_{w_b} = 0$ (bias fixed at $b = -0.2$, not a state in the true system — a **model mismatch**). The filter does *not* know acceleration is sinusoidal; it uses the same AR(1) model with $\phi = 0.9$.

$$\underbrace{\begin{bmatrix} x_{k+1} \\ v_{k+1} \\ a_{k+1} \\ b_{k+1} \end{bmatrix}}_{\boldsymbol{\chi}_{k+1}} = \underbrace{\begin{bmatrix} 1 & T_s & 0 & 0 \\ 0 & 1 & T_s & 0 \\ 0 & 0 & \phi & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}}_{\boldsymbol{\Phi}_{\text{KF}},\ \text{state transition}} \underbrace{\begin{bmatrix} x_k \\ v_k \\ a_k \\ b_k \end{bmatrix}}_{\boldsymbol{\chi}_k} + \underbrace{\begin{bmatrix} 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & \sigma_{w_a}^2 & 0 \\ 0 & 0 & 0 & 0 \end{bmatrix}}_{Q,\ \text{process noise } (\sigma_{w_b}=0)} \underbrace{\begin{bmatrix} 0 \\ 0 \\ \bar{w}_{a,k} \\ 0 \end{bmatrix}}_{\bar{\mathbf{w}}_k}$$

$$\begin{aligned} &\leftarrow x_{k+1} = x_k + T_s v_k \quad \text{(Euler position)} \\ &\leftarrow v_{k+1} = v_k + T_s a_k \quad \text{(Euler velocity)} \\ &\leftarrow a_{k+1} = \phi\, a_k + \sigma_{w_a} \bar{w}_{a,k} \quad \text{(AR(1))} \\ &\leftarrow b_{k+1} = b_k \quad \text{(bias fixed, no noise)} \end{aligned}$$

Acceleration input, sum of two sinusoid

$$a_k^{\text{input}} = \sin(\omega_1 t) + 0.7 \sin(\omega_2 t), \qquad \omega_1 = 2\pi \cdot 0.1 \text{ rad/s}, \quad \omega_2 = 2\pi \cdot 1 \text{ rad/s}$$

Model mismatch: sinusoidal input vs. AR(1) model.
The filter models acceleration as AR(1) with $\phi = 0.9$, giving a time constant $\tau = -T_s / \ln(\phi) \approx 10\,\text{ms}$ — meaning the **model expects acceleration to decorrelate in ∼10 steps**.

The true input however contains a 0.1 Hz component **correlated over thousands of steps** and a 1 Hz component **correlated over hundreds**.

The slow 0.1 Hz sinusoid in particular resembles a drifting bias to the AR(1) filter — it cannot distinguish a gradual rise/fall in acceleration from a shifting bias. Some sinusoidal variation therefore leaks into $\hat{b}$, causing the bias estimate to oscillate persistently around -0.2 rather than locking in.

Here the model mismatch is the source of the contamination.

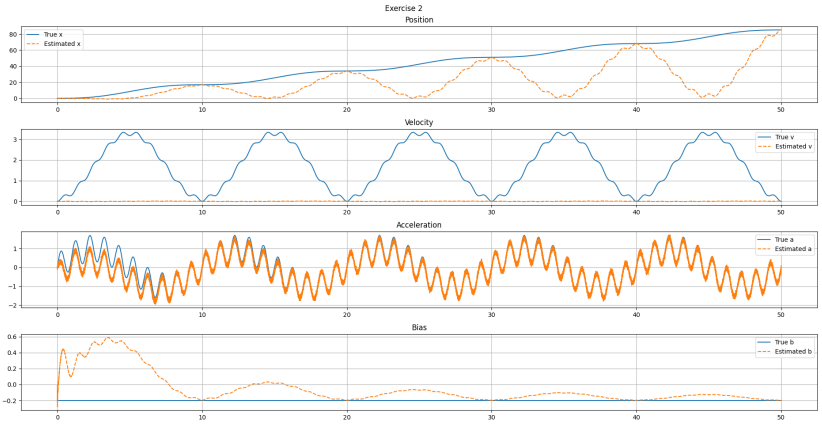

Figure 4: Python simulation output. **Blue** = true signal; **Red** = KF estimate. Parameters: $\phi = 0.9$, $\sigma_{w_a} = \sigma_\nu = 0.1$, $\sigma_{w_b} = 0.0$, $b = -0.2$, $T_s = 0.001$, $N = 50\,000$ steps $(50\,\mathrm{s})$.

Exercise 4 Expand to 2D

Built on Exercise 2 with external acceleration input, now expanded to two independent axes.

True system — 6 states $[x,\ v^x,\ a^x,\ y,\ v^y,\ a^y]$:

$$\underbrace{\begin{bmatrix} x_{k+1} \\ v^x_{k+1} \\ a^x_{k+1} \\ y_{k+1} \\ v^y_{k+1} \\ a^y_{k+1} \end{bmatrix}}_{\boldsymbol{\chi}_{\text{true},k+1}} = \underbrace{\begin{bmatrix} 1 & T_s & 0 & 0 & 0 & 0 \\ 0 & 1 & T_s & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & T_s & 0 \\ 0 & 0 & 0 & 0 & 1 & T_s \\ 0 & 0 & 0 & 0 & 0 & 0 \end{bmatrix}}_{\boldsymbol{\Phi}_{\text{true}},\ \text{state transition}} \underbrace{\begin{bmatrix} x_k \\ v^x_k \\ a^x_k \\ y_k \\ v^y_k \\ a^y_k \end{bmatrix}}_{\boldsymbol{\chi}_{\text{true},k}} + \underbrace{\begin{bmatrix} 0 & 0 \\ 0 & 0 \\ 1 & 0 \\ 0 & 0 \\ 0 & 0 \\ 0 & 1 \end{bmatrix}}_{\boldsymbol{\Gamma}} \underbrace{\begin{bmatrix} a^x_k \\ a^y_k \end{bmatrix}}_{\text{known inputs}}$$

$$\begin{aligned} \leftarrow x_{k+1} &= x_k + T_s v^x_k \\ \leftarrow v^x_{k+1} &= v^x_k + T_s a^x_k \\ \leftarrow a^x_{k+1} &= a^{x,\text{input}}_k \\ \leftarrow y_{k+1} &= y_k + T_s v^y_k \\ \leftarrow v^y_{k+1} &= v^y_k + T_s a^y_k \\ \leftarrow a^y_{k+1} &= a^{y,\text{input}}_k \end{aligned}$$

From $\boldsymbol{\Phi}_{\text{true}}$ it is clear that the two axes evolve independently — no cross-axis coupling. This is intentional: misalignment between axes is a **deterministic** error (captured by M_a in the full sensor model) and is assumed to be calibrated away beforehand, not handled by the Kalman filter.

Kalman filter — 8 states $[x,\ v^x,\ a^x,\ b^x,\ y,\ v^y,\ a^y,\ b^y]$:

Measurement equation: $y_k = H\,\boldsymbol{\chi}_k + R^{1/2}\bar{\nu}_k$

$$\underbrace{\begin{bmatrix} x_{k+1} \\ v^x_{k+1} \\ a^x_{k+1} \\ b^x_{k+1} \\ y_{k+1} \\ v^y_{k+1} \\ a^y_{k+1} \\ b^y_{k+1} \end{bmatrix}}_{\boldsymbol{\chi}_{k+1}} = \underbrace{\begin{bmatrix} 1 & T_s & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & T_s & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & \phi & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & T_s & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 & T_s & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & \phi & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 \end{bmatrix}}_{\boldsymbol{\Phi}_{\text{KF}},\ \text{state transtion}} \underbrace{\begin{bmatrix} x_k \\ v^x_k \\ a^x_k \\ b^x_k \\ y_k \\ v^y_k \\ a^y_k \\ b^y_k \end{bmatrix}}_{\boldsymbol{\chi}_k} + \underbrace{\begin{bmatrix} 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & \sigma^2_{w^x_a} & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & \sigma^2_{w^x_b} & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & \sigma^2_{w^y_a} & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & \sigma^2_{w^y_b} \end{bmatrix}}_{Q,\ \text{process noise}}{}^{1/2} \underbrace{\begin{bmatrix} 0 \\ 0 \\ \bar{w}^x_{a,k} \\ 0 \\ 0 \\ 0 \\ \bar{w}^y_{a,k} \\ 0 \end{bmatrix}}_{\bar{\mathbf{w}}_k}$$

State equation: $\boldsymbol{\chi}_{k+1} = \boldsymbol{\Phi}\,\boldsymbol{\chi}_k + Q^{1/2}\bar{\mathbf{w}}_k$ Each row of $\boldsymbol{\Phi}$ encodes one update rule:

$$\underbrace{\begin{bmatrix} y^x_k \\ y^y_k \end{bmatrix}}_{\mathbf{y}_k} = \underbrace{\begin{bmatrix} 0 & 0 & 1 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 1 & 1 \end{bmatrix}}_{H,\ \text{measurement matrix}} \underbrace{\begin{bmatrix} x_k \\ v^x_k \\ a^x_k \\ b^x_k \\ y_k \\ v^y_k \\ a^y_k \\ b^y_k \end{bmatrix}}_{\boldsymbol{\chi}_k} + \underbrace{\begin{bmatrix} \sigma^2_{\nu^x} & 0 \\ 0 & \sigma^2_{\nu^y} \end{bmatrix}}_{R,\ \text{measurement noise}}{}^{1/2} \underbrace{\begin{bmatrix} \bar{\nu}^x_k \\ \bar{\nu}^y_k \end{bmatrix}}_{\bar{\boldsymbol{\nu}}_k}$$

$$\begin{aligned} \leftarrow y^x_k &= a^x_k + b^x_k + \sigma_{\nu^x}\bar{\nu}^x_k \\ \leftarrow y^y_k &= a^y_k + b^y_k + \sigma_{\nu^y}\bar{\nu}^y_k \end{aligned}$$

Same assumes here also that two axes evolve independently — no cross-axis coupling. Calibrated away beforehand, not handled by the Kalman filter.

Syntax	What it does
`A @ B`	Matrix multiplication of `A` and `B`. Equivalent to `np.matmul(A, B)`. Works for 2-D arrays (matrix × matrix), 2-D × 1-D (matrix × vector), and 1-D × 1-D (dot product). Example: `H @ chi_hat` multiplies the (1×4) matrix `H` by the $(4,)$ vector, giving a scalar.
`np.eye(n)`	Returns the $n \times n$ **identity matrix** — ones on the diagonal, zeros everywhere else. `np.eye(4)` gives the 4×4 identity used to initialise $P = I$ ("equal, moderate uncertainty in all states, no correlations").
`np.zeros(n)`	Returns a 1-D array of `n` zeros. `np.zeros(4)` gives `[0., 0., 0., 0.]` — used to initialise `chi_hat` (zero starting guess for all four states).
`np.zeros(N)` (storage)	Same call, but used to pre-allocate an empty array of length N to store results in the loop. Pre-allocating is faster than appending inside a loop.
`np.diag([...])`	Creates a square **diagonal matrix** from a list. `np.diag([0, 0, 0.01, 0])` puts those values on the diagonal and zeros elsewhere — used to build Q.
`np.full(N, val)`	Returns an array of length N where every entry is `val`. `np.full(N, -0.2)` creates the constant true-bias array for plotting.
`np.random.randn()`	Draws **one** sample from $\mathcal{N}(0, 1)$ (standard normal: zero mean, unit variance).
`np.random.randn(N)`	The argument `N` sets the **number of samples** returned — gives a 1-D array of `N` independent draws from $\mathcal{N}(0, 1)$. Used to generate all N measurement noise samples at once rather than calling `randn()` inside the loop.
`np.random.seed(s)`	Fixes the random number generator to seed `s` so the simulation produces the same random values every run. Remove or change `s` to get a different realization.
`np.linalg.inv(A)`	Computes the **matrix inverse** A^{-1}. Used to compute S_k^{-1} in the Kalman gain. For a scalar S_k this is just $1/S_k$.
`.flatten()`	Collapses any array to 1-D. After `K @ inn` the result can be shape $(4, 1)$; `.flatten()` converts it to $(4,)$ so it can be added directly to `chi_hat`.
`.T`	**Transpose** of a 2-D array. `H.T` turns the (1×4) row vector into a (4×1) column vector.

Kalman Filter Equations — Don't go say, don't think have time

---

Initialisation

$$\hat{\boldsymbol{\chi}}_{0|-1} = \mathbf{0}, \qquad P_{0|-1} = I$$

$\hat{\boldsymbol{\chi}}_{0|-1} = \mathbf{0}$ is a neutral starting guess — all four states (position, velocity, acceleration, bias) set to zero because we have no prior knowledge.

$P_{0|-1} = I$ sets the initial error covariance to the identity matrix. The **diagonal entries of P are the variances of each state estimate**

> A larger diagonal entry means higher uncertainty (larger variance) for that state. Setting $P_{0|-1} = I$ is a moderate, equal initial guess: equal uncertainty across all four states and no assumed correlation between them (off-diagonal entries zero). Both values are overwritten rapidly once measurements arrive.

Step A — Measurement update

Runs every step as soon as a new sensor reading y_k arrives. The goal: correct the prediction from the previous step using what the sensor just observed.

1. Innovation

$$\tilde{y}_{k|k-1} = y_k - H\hat{\boldsymbol{\chi}}_{k|k-1}$$

could you add underbrace here to say predicted measurement under H and x

y_k is a **scalar** — a single accelerometer reading at step k. This the acceleration measurement $y_k = a_k + b_k + \nu_k$. Not the True measurement So true acceleration plus bias plus process/sensor noise

$\hat{\boldsymbol{\chi}}_{k|k-1}$ is the **predicted state** from the previous step, a 4×1 vector $[\hat{x},\ \hat{v},\ \hat{a},\ \hat{b}]^\top$.

$H = [0,\ 0,\ 1,\ 1]$ is the **measurement matrix**: it projects the 4-state vector down to a scalar prediction of what the sensor would read.

A large innovation means the measurement surprised the filter; a small innovation means the prediction was already accurate.

> H come directly from the **sensor physics** — they are not a design choice. For the accelerometer the physics says it reads exactly $1{\cdot}a_k + 1{\cdot}b_k$, so the corresponding entries are 1 and 1. The zeros for position and velocity are also physics: they genuinely do not appear in what an accelerometer outputs.

> $H\hat{\boldsymbol{\chi}}_{k|k-1}$ collapses the 4-state vector to a scalar: $0{\cdot}\hat{x} + 0{\cdot}\hat{v} + 1{\cdot}\hat{a} + 1{\cdot}\hat{b} = \hat{a}_{k|k-1} + \hat{b}_{k|k-1}$ — the filter's prediction of what the sensor would read.

2. Innovation covariance

$$S_k = H\,P_{k|k-1}\,H^\top + R$$

- $HP_{k|k-1}H^\top$ — the state prediction uncertainty *projected into measurement space* via H.

  The H matrices on each side extract only the acceleration/bias rows and columns of P, convertes state uncertainty into a scalar uncertainty.

- $R = \sigma_\nu^2$ — the sensor noise variance; the irreducible noise on every individual reading.

Together they give the total uncertainty of the innovation, which is used in the next step as a normalising denominator.

S_k is the total variance of the innovation — how uncertain we should be about $\tilde{y}_{k|k-1}$ itself.

3. Kalman gain

$$K_k = P_{k|k-1}\,H^\top\,S_k^{-1} \qquad \Longleftrightarrow \qquad K_k = \frac{\overbrace{P_{k|k-1}H^\top}^{\text{prediction uncertainty (in measurement space)}}}{\underbrace{S_k}_{\text{total uncertainty}}}$$

K_k is a 4×1 vector controlling how much each state is corrected by the innovation. It is the **ratio of prediction uncertainty to total uncertainty**:

- **Numerator** $P_{k|k-1}H^\top$: how uncertain the model prediction is, projected into measurement space.
- **Denominator** $S_k = HP_{k|k-1}H^\top + R$: that same prediction uncertainty plus sensor noise R.

A **high** K_k means the model is uncertain relative to the sensor — trust the measurement more. A **low** K_k means the sensor is noisy relative to the model — trust the prediction more.

> In the general case $S_k \in \mathbb{R}^{m\times m}$ and $K_k \in \mathbb{R}^{n\times m}$, where m is the number of measurements and n the number of states. Here y_k is a scalar (position only), so $m = 1$ and S_k collapses to a scalar.
>
> $P_{k|k-1}H^\top$ uses only $H^\top$ (4×1), giving a 4×1 vector, whereas step 2. $S_k = HP_{k|k-1}H^\top + R$ sandwiches with both H and $H^\top$, which collapses the result to a scalar.
>
> Therefore $K_k = (4 \times 1)/(1 \times 1)$ remains a 4×1 vector — one gain per state.

4. State update

$$\hat{\boldsymbol{\chi}}_{k|k} = \hat{\boldsymbol{\chi}}_{k|k-1} + K_k\,\tilde{y}_{k|k-1}$$

$\hat{\boldsymbol{\chi}}_{k|k-1}$ is the prediction (built from all data up to step $k-1$).

The result $\hat{\boldsymbol{\chi}}_{k|k}$ is now the best estimate at step k after having seen y_k — **uncertainty decreases**.

5. Covariance update (Joseph form)

$$P_{k|k} = (I - K_k H)\, P_{k|k-1}\, (I - K_k H)^\top + K_k R K_k^\top$$

After seeing the measurement, the filter knows more about the state, so P shrinks — **uncertainty decreases**.

The factor $(I - K_k H)$ represents how much uncertainty is *not* explained by the measurement: if $K_k H = I$ (perfect observation of all states) the whole prior uncertainty is cancelled.

The extra $K_k R K_k^\top$ adds back the small residual uncertainty introduced by the sensor noise itself.

> The symmetric sandwich form $(I - K_k H)\, P\, (I - K_k H)^\top$ is used instead of the simpler $(I - K_k H)P$ because it keeps P symmetric and positive-definite even after thousands of steps of floating-point arithmetic — a numerical stability guarantee.

Step B — Time update (prediction)

Runs immediately after Step A to prepare the estimate for the next step.

1. State prediction

$$\hat{\boldsymbol{\chi}}_{k+1|k} = \boldsymbol{\Phi}\, \hat{\boldsymbol{\chi}}_{k|k}$$

$\boldsymbol{\Phi}$ is the **model's approximation** of how the true system evolves — it is not the exact true transition, just the best mathematical description.

Multiplying the corrected estimate $\hat{\boldsymbol{\chi}}_{k|k}$ by $\boldsymbol{\Phi}$ rolls it one step forward. The output $\hat{\boldsymbol{\chi}}_{k+1|k}$ is the predicted state for the next step, ready to be corrected again when y_{k+1} arrives.

2. Covariance prediction

$$P_{k+1|k} = \boldsymbol{\Phi}\, P_{k|k}\, \boldsymbol{\Phi}^\top + Q$$

$\boldsymbol{\Phi}\, P_{k|k}\, \boldsymbol{\Phi}^\top$ propagates the current uncertainty forward through the dynamics model — the same way $\boldsymbol{\Phi}$ propagates the state estimate.

Q is then **added** on top. It is the noise process noise — it is the covariance of the random **kick** each state takes during the transition itself.

$Q = \mathrm{diag}(0,\, 0,\, \sigma_{w_a}^2,\, \sigma_{w_b}^2)$ is the covariance matrix of those kicks — it describes *how wide the distribution of the next state is* given the current one, not anything a sensor reads.

The larger Q is, the more the filter admits it does not know exactly where the state will land next step, and the faster P grows.

uncertainty grows during prediction and is only brought back down when the next measurement arrives in Step A.

Worked example for $HPH^\top$ with $H = [0, 0, 1, 1]$ and a concrete P. Blue marks the rows H picks from P; red marks the columns $H^\top$ picks from HP.

$$P = \begin{bmatrix} 0.5 & 0 & 0 & 0 \\ 0 & 0.4 & 0 & 0 \\ 0 & 0 & 0.3 & 0.1 \\ 0 & 0 & 0.1 & 0.2 \end{bmatrix}$$

Step 1 — HP: $[0, 0, 1, 1]$ multiplies P, adding row 3 + row 4:

$$HP = \begin{bmatrix} 0{+}0, & 0{+}0, & 0.3{+}0.1, & 0.1{+}0.2 \end{bmatrix} = \begin{bmatrix} 0, & 0, & 0.4, & 0.3 \end{bmatrix}$$

Step 2 — $(HP)H^\top$: dot with $[0, 0, 1, 1]^\top$, picking columns 3 and 4 of HP:

$$HPH^\top = 0{\cdot}0 + 0{\cdot}0 + 0.4{\cdot}1 + 0.3{\cdot}1 = 0.7$$

So the result is the sum of the **four entries in the a–b corner of P**:

$$HPH^\top = \underbrace{P_{33}}_{0.3} + \underbrace{P_{34}}_{0.1} + \underbrace{P_{43}}_{0.1} + \underbrace{P_{44}}_{0.2} = 0.7$$

The combined uncertainty about $a + b$ is larger than adding their variances alone. The position and velocity rows/columns vanish entirely because of the zeros in H — their uncertainty is irrelevant to what the accelerometer reads.